DPUMMI

Dies Simulation of graphic man-machine interfaces

1. Inputs:

Inputs are generated by MMI elements (see below).

2. Static parameters:

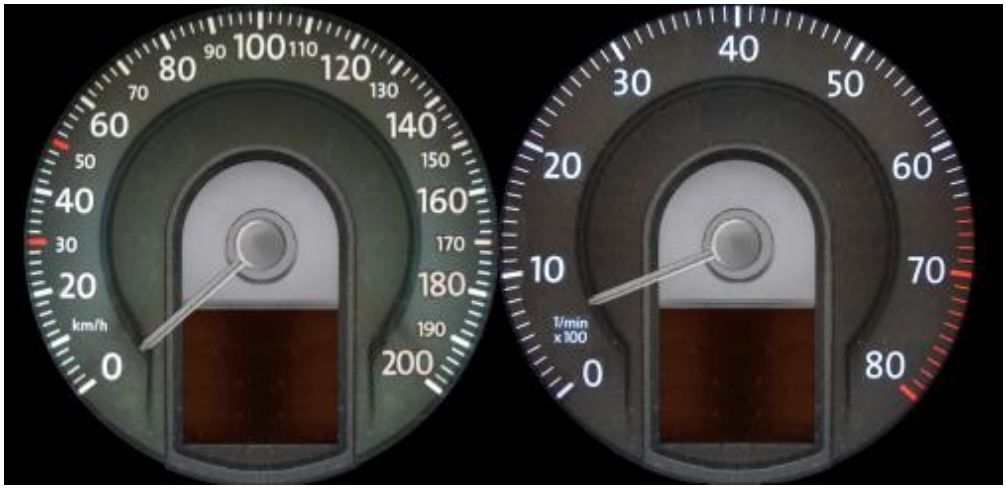
WindowX	X coordinate of the MMI window (in screen coordinates)
WindowY	Y coordinate of the MMI window (in screen coordinates)
WindowW	Width of the MMI window (in pixels)
WindowH	Height of the MMI window (in pixels)
WindowTopmost	Defines whether the display window should always be docked "in the foreground" (similar as the Windows task bar). The default value is false.
W	Width of the MMI coordinate systems. All position and size data of MMI elements refer to this coordinate system. The origin is in the bottom left-hand corner of the MMI window.
H	Height of the MMI coordinate system. All position and size data of MMI elements refer to this coordinate system. The origin is in the bottom left-hand corner of the MMI window.
MouseReleaseDelayMS	Delays mouse release events by the given time (in milliseconds). The default value is 0, i.e. no delay. The value can be increased (e.g. to 30) if Button elements are found to not work correctly. This happens with some touchscreen drivers (e.g. the Microsoft default driver), which send mouse press events at the same time as mouse release events (when the finger is lifted from the touchscreen).

3. Outputs:

Outputs are generated by MMI elements (see below).

How it works:

MMIs such as the instrument cluster shown in the illustration below can be simulated with DPUMMI.



The following basic elements are available:

ImageDisplay	5
Group	6
TextDisplay	6
VarTextDisplay	7
NumberDisplay	8
Light	9
Button	9
Needle	10
CircularRegulator	11
LinearRegulator	12
LinearProgressBar	12
Scope	13

Element-specific parameters are used to define the displayed image of the elements. For example, in order to display the image, the position and size as well as a list of bitmap graphics is required. Element-specific inputs are used to switch elements and outputs may be used as input of other DPUs. For example, the ImageDisplay may provide an input In of type unsigned int, that indicates the index of the displayed image from the bitmap graphics list. The parameters, the inputs and outputs of the available elements are described in section "Element types".

Each basic element may function as a container that incorporates additional elements. The resulting hierarchy in this scenario creates complex MMI elements that are composed of simple elements (e.g., fuel indicator with warning light). The structure of such a hierarchy is described in the following chapter "Configuration".

Configuration:

The MMI is defined in the configuration file in DPUMMI section 'Instance':

Grammatic 1a MMI definition in DPUMMI

(within a DPU pool:)

```
DPUMMI <MMI instance name>
{
```

```
      Computer = { (,)<computer name> };
      Index = <number>;
      (0...*)<initialization of parameters>
      (0…1)<MMI definition (see Grammatic 2 below)>
};
```

Please note: Starting with SILAB Version 2.2, several instances of DPUMMI can be executed on the same simulation computer. Each instance operates independently from the other, i.e., it uses its own display window and sets up separate DPU inputs and outputs.

As an alternative definition of the MMI within the DPUMMI instance, the MMI can also be defined in the global configuration segment "**MMI** *<MMI instance name>*"(on the top level of the test configuration).

Grammatic 1b global MMI definition

(at top level of configuration:)

```
MMI <MMI instance name>
{
      (0...*)<initialization of parameters>
      (1)<MMI-definition (see Grammatic 2 below)>
};
```

This global configuration segment can be listed several times. All elements contained in these segments are generated together.

The segments will be processed in sequence of their occurrence in the configuration.

Since an < initialization of parameters >, e.g. the W parameter, may occur in the second or following segments, the MMI can be built up in several steps and expanded subsequently when using global configuration segments.

The hierarchy of the MMI elements is given in the <MMI-definition> segment. Similar as in the DPUs, instances of certain element types are generated. On each hierarchy level, elements can be linked in a <connections of elements> segment.

Grammatic 2 MMI definition structure

```
<MMI-definition> 
(1...*)<MMI container>

<MMI container> 
      <element type> <container name>
      {
            <element parameters>
            (0...*)<MMI container>
      };
      (0,1)<connections of elements>
```

```
<connections of elements>
      Connections =
      {
            (,)<connection>
      };

<connection>
      <container path>.<output> -> <container path>.<input>
```

The *<container path>* in a *<connection>* consists of a series of container names that are separated by dots. Starting with the container in which the *<connection of elements>* segment can be found, the path to the subordinate containers is specified. The following simple example explains this procedure:

```
DPUMMI mmi
{
      ...
      ImageDisplay image1
      {
            ...
            ImageDisplay image2
            {
                  ...
                  Light l1
                  {
                        ...
                  };
                  Light l2
                  {
                        ...
                  };
            };

            Button b
            {
                  ...
            };

            Connections =
            {
                  b.Out -> image2.l1.In,
                  b.Out -> In
            };
      };
};
```

In this example, the output *Out* of button *b* is used to actuate the light *l1* that is located in container *image2* and the display screen *image1*.

The link of MMI elements with inputs and outputs of other DPUs is described in section "DPU Configuration" of document "SILAB Configuration and Operation". The inputs and outputs of the MMI elements in the MMI-DPU instance will be addressed in the form of

<instance name of MMI-DPU>.<container path>.<input/output>

If the input *In* of light *I2* in the example above must be activated by an output Warning of a DPU instance acc then the respective link is written as follows:

```
Connections =
{
      acc.Warning -> mmi.image1.l2.In
};
```

Element types

The following list describes the available element types and their respective parameters and inputs/outputs.

Note: Detailed information pertaining to coordinate system applicable to the parameters X, Y, W, and H can be found at the end of the section "Configuration systems".

ImageDisplay

Displays a certain image from a list of images

Parameters:

X *Y*	Position of the lower left-hand corner of the display in the incorporated container. The origin of a container is in its lower left-hand corner.
W	Width of the ImageDisplay.
H	Height of the ImageDisplay.
Images	Tupel with file name of images in PNG or TGA format. The images must be stored with 16 million colors. Transparent areas of the image represent 0 values in the alpha channel; the alpha value 1 is allocated to opaque areas. The images will be searched in directory *SILAB\lib\graphics\default\mmi* or in the project directory. When entering file names, directories may be specified that are subdirectories of the project directory or *SILAB\lib\graphics\default\mmi*.

Inputs:

In	Integer that specifies the zero-based index of the image in the Images list that is shown.
Show	Boolean value (true or false), indicates whether the selected image is displayed. The default value is true.

Example:

```
ImageDisplay image
{
      X = 10;
      Y = 10;
      W = 300;
      H = 200;
      Images = ("acc/empty.png", "acc/collision.png", "acc/warning.png");
};
```

The images shown in the example are loaded from the directory *SILAB\lib\graphics\default\mmi\acc*.

Group

An element used only for logical grouping purposes of other elements while not being displayed itself.

Parameters:

X, Y, W, H	Same as in the ImageDisplay.

Inputs:

Show	Boolean value (true or false) that indicates whether all subordinate elements should be displayed. The default value is true.

TextDisplay

Displays a certain text from a list of texts

Parameters:

X, Y, W, H	Same as in the ImageDisplay. The height of the text will be adjusted to H, its width will be adjusted proportionally.
Font	Name of the TrueType Font file in quotation marks ("<fontfile-name>"). If this parameter is not specified, "arial.ttf" will be applied. When indicating a relative path, the system searches in the Windows font directory, \SILAB\lib\graphics\default\mmi and the project directory. The specification of an absolute path is also possible.
FontColor	This tuple defines the color of the font. This tuple uses the format (<r>,<g>,<b>), whereby <r>, <g> and <b> range from 0 to 1.
AntiAliasing	Defines whether anti-aliasing (smoothing the jagged appearance) should be used. The following values are possible: "on", "off", "true", "false". The default is "on".
Alignment	The horizontal adjustment of the text. Possible values are "left", "right" and “center”.
Texts	List of texts.

Inputs:

In	Integer that specifies the zero-based index of the text in the Texts list that is shown.
Show	Boolean value (true or false) that indicates whether the text field is displayed. The default value is true.

Example:

```
TextDisplay text
{
      X = 10;
      Y = 10;
      W = 300;
      H = 200;
      Font = "verdanab.ttf";
      FontColor = (1.0, 0.0, 0.0);
      Texts = ("Danger of black ice!", "Check oil level!", "Service!");
};
```

VarTextDisplay

Displays a dynamically variable text. The text will be specified directly via a DPU input as a string variable.

Parameters:

X, *Y*, *W*, *H*	Same as in the TextDisplay.
Font	Name of the TrueType Font file in quotation marks ("<fontfile-name>"). If this parameter is not specified, "arial.ttf" will be applied. When indicating a relative path, the system searches in the Windows font directory, \SILAB\lib\graphics\default\mmi and the project directory. The specification of an absolute path is also possible.
FontColor	This tuple defines the color of the font. This tuple uses the format (<r>,<g>,<b>), whereby <r>, <g> and <b> range from 0 to 1.
AntiAliasing	Defines whether anti-aliasing (smoothing the jagged appearance) should be used. The following values are possible: "on", "off", "true", "false". The default is "on".
Alignment	The horizontal adjustment of the text. Possible values are "left", "right" and “center”.

Inputs:

In	Integer that specifies the zero-based index of the text in the Texts list that is shown.
Show	Boolean value (true or false) that indicates whether the text field is displayed. The default value is true.

Example:

```
VarTextDisplay vartext
{
      X = 10;
      Y = 10;
```

```
        W = 300;
        H = 200;
        Font = "verdanab.ttf";
        FontColor = (1.0, 0.0, 0.0);
};
```

NumberDisplay

Displays a specified numeric value (e.g. selected ACC speed)

Parameters:

X, Y, W, H	Same as in the TextDisplay.
Font	Name of the TrueType Font file in quotation marks ("<fontfile-name>"). If this parameter is not specified, "arial.ttf" will be applied. When indicating a relative path, the system searches in the Windows font directory, \SILAB\lib\graphics\default\mmi and the project directory. The specification of an absolute path is also possible.
FontColor	This tuple defines the color of the font. This tuple uses the format (<r>,<g>,<b>), whereby <r>, <g> and <b> range from 0 to 1.
AntiAliasing	Defines whether anti-aliasing (smoothing the jagged appearance) should be used. The following values are possible: "on", "off", "true", "false". The default is "on".
Alignment	The horizontal adjustment of the text. Possible values are "left", "right" and “center”.
Format	Indicates the format of the displayed text. Any format string as in the C-function printf() can be used. The number (here it is a double value) will be entered as the first parameter. Therefore, it may be entered into the text as %f or %g. When inputting %.0f or %.0g decimal places will not be displayed.

Example (assuming the input value is 21.38):

Format string		**displayed text**
"%.0f km/h"	→	21km/h
"The value is %f."	→	The value is 21.38.

Inputs:

In	Floating point number which current value is entered into the format text and displayed.
Show	Boolean value (true or false) that indicates whether the text field is displayed. The default value is true.

Example:

```
NumberDisplay ACCSetSpeed
{
        X = 140;
        Y = 140;
        W = 423;
```

```
        H = 23;
        FontColor = (0.98, 0.322, 0);
        Format    = "ACC %3.0f km/h";   # format string in C-syntax
        Font = "verdanab.ttf";
};
```

Light

Simulates a light.

Parameters:

X, *Y*, *W*, *H*	Same as in the ImageDisplay.
OnImage	File name of the image that will be displayed when light is illuminated. Documentation of ImageDisplay applies for this file name as well.
OffImage	File name of the image that will be displayed if the light is off. Documentation of ImageDisplay applies for this file name as well.

Inputs:

In	Boolean value that indicates whether the light is on (value 1) or off (value 0)

Example:

```
Light l
{
        X = 50;
        Y = 50;
        W = 100;
        H = 50;
        OnImage = "acc\system_on.png";
        OffImage = "acc\system_off.png";
};
```

NB: This Light can be done as an ImageDisplay with the following configuration:

```
ImageDisplay l
{
        X = 50;
        Y = 50;
        W = 100;
        H = 50;
        Images = ("acc\system_off.png", "acc\system_on.png");
};
```

Button

Simulates a button or switch.

Parameters:

X, *Y*, *W*, *H*	Same as in the ImageDisplay.
Style	One of the following designators: Button: This is a switch. PushButton: This is a push button.
PressedImage	File name of the images that indicates when the button/switch is activated. Documentation of ImageDisplay applies for this file name as well.
UnpressedImage	File name of the images that indicates when the button/switch is not activated. Documentation of ImageDisplay applies for this file name as well.

Outputs:

Out	Boolean value that indicates whether the button/switch is activated (value 1) or not (value 0).

Example:

```
Button OnOff
{
      X = 30;
      Y = 20;
      W = 50;
      H = 50;
      Style = Button;
      PressedImage = "acc\flat.png";
      UnpressedImage = "acc\raised.png";
};
```

Needle

Simulates the needle of a circular instrument. As a rule, needles of a circular instrument are used in containers of the ImageDisplay type. The ImageDisplay provides the scale image of the circular instrument.

Parameters: (see also Figure 1)

X *Y*	Position of the lower left-hand corner of the needle element in the incorporated container. The origin of a container is in its lower left-hand corner.
W	Width of the background image (e.g., tachometer disc).
H	Height of the background image (e.g., tachometer disc).
XMount *YMount*	Mounting point relative to the lower left-hand corner of the needle element.
XNeedleMount *YNeedleMount*	Mounting point on the needle, relative to the lower left-hand corner of the needle image.
AngleMin	Minimum angle of the needle (0° = 9 o'clock position).

ValueMin	Input value associated with AngleMin.
AngleMax	Maximum angle of the needle.
ValueMax	Input value associated with AngleMax.
NeedleImage	File name of the needle image. Documentation of ImageDisplay applies for this file name as well. The needle provided in the image must be vertical, meaning in the 12 o'clock position.

Inputs:

In	Double value between ValueMin and ValueMax.

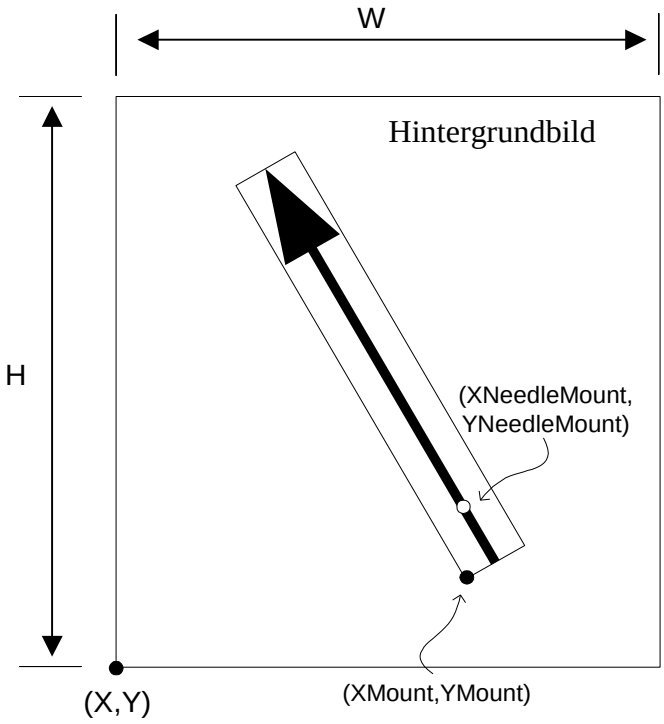


Figure 1: Parameter of the needle element type

CircularRegulator

Simulates a circular regulator. As a rule, circular regulators are used in containers of the ImageDisplay type. The ImageDisplay provides the scale image of the circular regulator.

Parameters: (see also Figure 1)

X	
Y	Position of the lower left-hand corner of the circular regulator element in the incorporated container. The origin of a container is in its lower left-hand corner.
W	Width of the circular regulator element.
H	Height of the circular regulator element.
XMount	

YMount	Mounting point relative to the lower left-hand corner of the circular regulator element.
XNeedleMount	
YNeedleMount	Mounting point relative to the circular regulator, relative to the lower left-hand corner of the needle image.
AngleMin	Minimum angle of the needle (0° = 9 o'clock position).
ValueMin	Input value associated with AngleMin.
AngleMax	Maximum angle of the needle.
ValueMax	Input value associated with AngleMax.

Outputs:

Out	Double value betweenValueMin and ValueMax.

LinearRegulator

Simulates a linear regualtor. As a rule, linear regulators are used in containers of the ImageDisplay type. The ImageDisplay provides the scale image of the linear regulator. Linear regulators have a finite number of conditions.

Parameters: (see also Figure 1)

X	
Y	Position of the lower left-hand corner of the linear regulator element in the incorporated container. The origin of a container is in its lower left-hand corner.
W	Width of the linear regulator element.
H	Height of the linear regulator element.
Alignment	Alignment of the linear regulator. Possible values are "horizontal" and "vertical".
ValueMin	Minimum initial value. Minimum value on the regulator's scale
ValueMax	Maximum initial value. Maximum value on the regulator's scale
Images	Tuple with file name of images in the PNG format. Documentation of ImageDisplay applies for this file name as well. Each image corresponds with a section on the regulator's scale. The number of sections is the same as those shown in the image. The size of all sections is identical. If the mouse or the touchscreen is used to activate a section, only the corresponding image will be displayed. At this moment, the initial value is identical with the lower value of the respective section on the regulator's scale.

Outputs:

Out	Double value betweenValueMin and ValueMax.

LinearProgressBar

Simulates a linear progress bar. As a rule, linear progress bars are used in containers of the ImageDisplay type. The ImageDisplay provides the scale image of the progress bar. Progress bars have a finite number of conditions.

Parameters: (see also Figure 1)

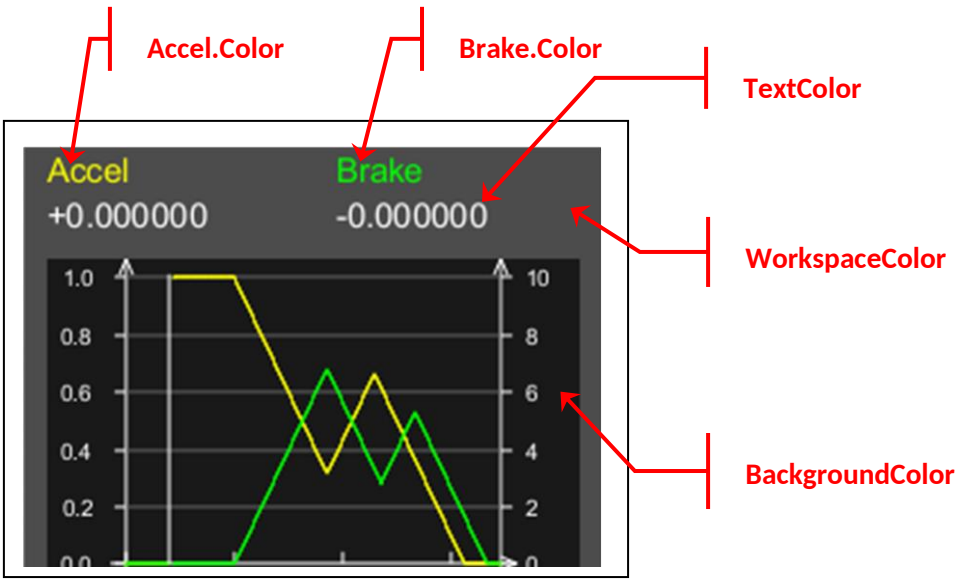
X *Y*	Position of the lower left-hand corner of the progress bar element in the incorporated container. The origin of a container is in its lower left-hand corner.
W	Width of the progress bar element.
H	Height of the progress bar element.
Alignment	Alignment of the progress bar. Possible values are "horizontal" and "vertical".
ValueMin	Minimum initial value. Minimum value on the progress bar
ValueMax	Maximum initial value. Maximum value on the progress bar
Images	Tuple with file name of images in the PNG format. Documentation of ImageDisplay applies for this file name as well. Each image corresponds with a section on the progress bar scale. The number of sections is the same as those shown in the image. The size of all sections is identical. If the value of an input In is located in a section, only the corresponding image will be displayed. **Caution**: Points in which the current image appears transparent, i.e. elements below translucent areas, cannot be clicked on.

Inputs:

In	Double value between ValueMin and ValueMax.

Scope (Oscilloscope display)

Simulates an oscilloscope with a variable Y1 and Y2 axis. Up to eight input channels can be displayed.



Parameters of the main display:

X, Y	Position of the lower left-hand corner of the display in the incorporated container. The origin of a container is in its lower left-hand corner.
W	Width of the scope display.

H	Height of the scope display.
Font	Name of the true-type font file in quotation marks ("<fontfile-name>"). If this parameter is not specified, "arial.ttf" will be applied. When indicating a relative path, the system searches in the Windows font directory, \SILAB\lib\graphics\default\mmi and the project directory. The specification of an absolute path is also possible.
TextHeight	Height of the text line. The scale of all other elements (e.g., axes) will be adjusted accordingly.
LineWidth	Width of the lines (axes, reference lines, etc.).
ShowButtons	Should buttons be displayed that allow the user to adjust the scale of the axes during the operating time of the simulation? ("true"/"false")
ShowDescription	Should the variable names and values be displayed? (Default value: "true"; "false" only shows the plot area without any titles.)
WorkspaceColor	Background color of the entire oscilloscope display ((r, g, b) or (r, g, b, a)).
BackgroundColor	Color of the main display area ((r, g, b) or (r, g, b, a)).
TextColor	Color of the text and axes ((r, g, b) or (r, g, b, a)).

Parameter of the individual channels:

Min	Minimum value of the channel (default: 0)
Max	Maximum value of the channel (default: 1)
AutoScale	Should the minimum and maximum value of the channel be adjusted automatically to the input data? ("true"/"false", default is "false".)
Color	Color of the channel ((r, g, b) or (r, g, b, a)).
Axe	Displays the channel of the nth Y axis (the values 1 and 2 are permitted). By default, the first channel is displayed on Y1 and the second channel is displayed on Y2.

Inputs:

In1 to In8	Data of the input channels.
Show	Boolean value (true or false) that indicates whether the selected image is displayed. The default value is true.

Example:

```
Scope DrivePars
{
      W = 256;
      H = 206;
      X = 0;
      Y = 0;
      WorkspaceColor = (0.3,0.3,0.3); # dark grey
      BackgroundColor = (0.1,0.1,0.1); # very dark grey
      TextColor = (1,1,1); # white
      ShowButtons = "false";

      define Channel Accel
```

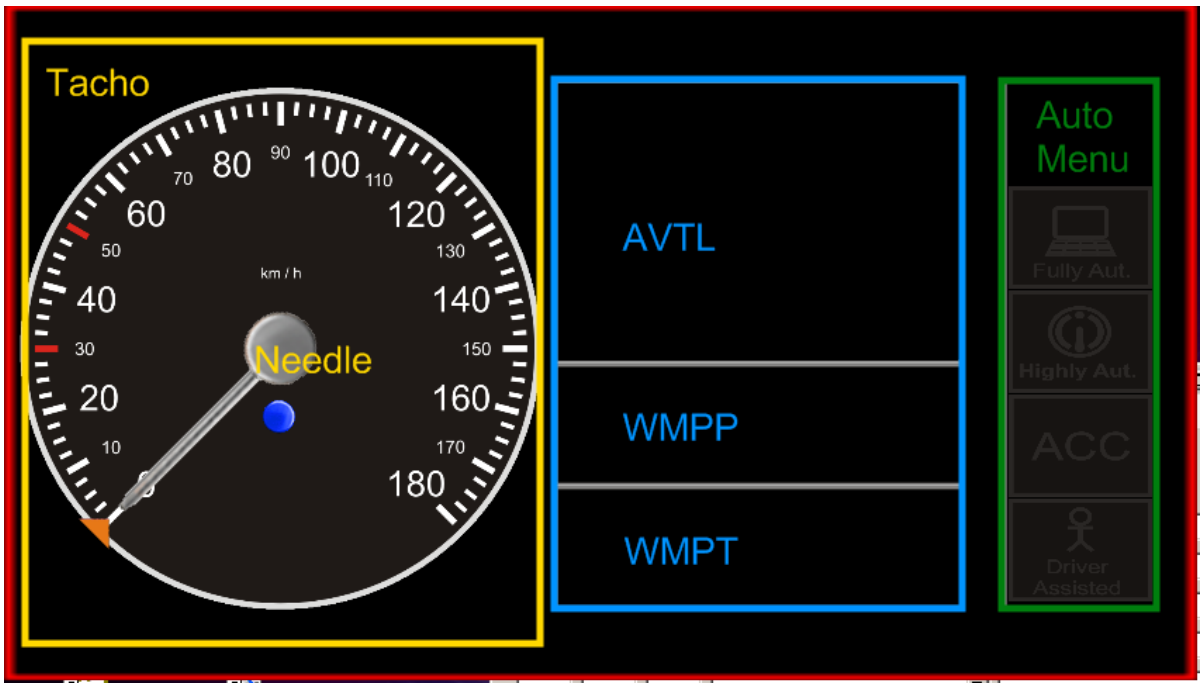
```
        {
                Color = (1,1,0);
                Min = 0;
                Max = 1;
        };

        define Channel Brake
        {
                Min = 0;
                Max = 10;
        };
};
```

This definition generates an oscilloscope that may be used to graphically display the current accelerator and brake pedal position. (In SILAB, the default value range of the accelerator pedal is 0 to 1; the default of the brake pedal is set at 0 to 10).

The inputs may be linked as follows:

```
Connections =
        Mockup.AcceleratorPedal -> MMI.DrivePars.In1,
        Mockup.BrakePedal -> MMI.DrivePars.In2
};
```

Coordinate system



To understand better which container and which value are responsible for what function, the above screenshot and the ("abridged") sample of the configuration will explain that.

Sample configuration:

```
DPUMMI MMI
{
	Computer = {HMI};
	Index = 500;
	WindowX = 0;
	WindowY = 0;
	WindowW = 848;
	WindowH = 480;
	W = 848;
	H = 480;

	ImageDisplay Black
	{
		X = 0;
		Y = 0;
		W = 848;
		H = 480;
		Images = ("mmi\sp3000_2\BackgroundBlack.png");
	};

	ImageDisplay Tacho
	{
		X = 10;
		Y = 50;
		W = 372;
		H = 372;
		Images = ("mmi\sp3000_2\Tacho.png");

		Needle Nadel
		{
			X = 0;
			Y = 0;
			W = 480;
			H = 441;
			XMount = 186;
			YMount = 186;
			XNeedleMount = 27;
			YNeedleMount = 27;
			AngleMin = -45;
			AngleMax = 225;
```

```
            ValueMin = 0;
            ValueMax = 180;
            NeedleImage = "mmi\sp3000_2\needle.png";
      };
};

ImageDisplay AVTL
{
      X = 390;
      Y = 224;
      W = 292;
      H = 203;
      Images = ("mmi\sp3000_2\ContainerVTL.png");

      ImageDisplay TargetHeadway
      {
            X = 79;
            Y = 10;
            W = 135;
            H = 95;
            Images = ("mmi\sp3000_2\targetheadway.png");
      };
};

ImageDisplay WMPP
{
      X = 390;
      Y = 137;
      W = 292;
      H = 89;
      Images = ("mmi\sp3000_2\ContainerWMPP.png");

      ImageDisplay vOK
      {
            X = 40;
            Y = 20;
            W = 56;
            H = 52;
            Images = ("mmi\sp3000_2\WMPPe.png");
      };

      ImageDisplay WMPT
      {
            X = 390;
```

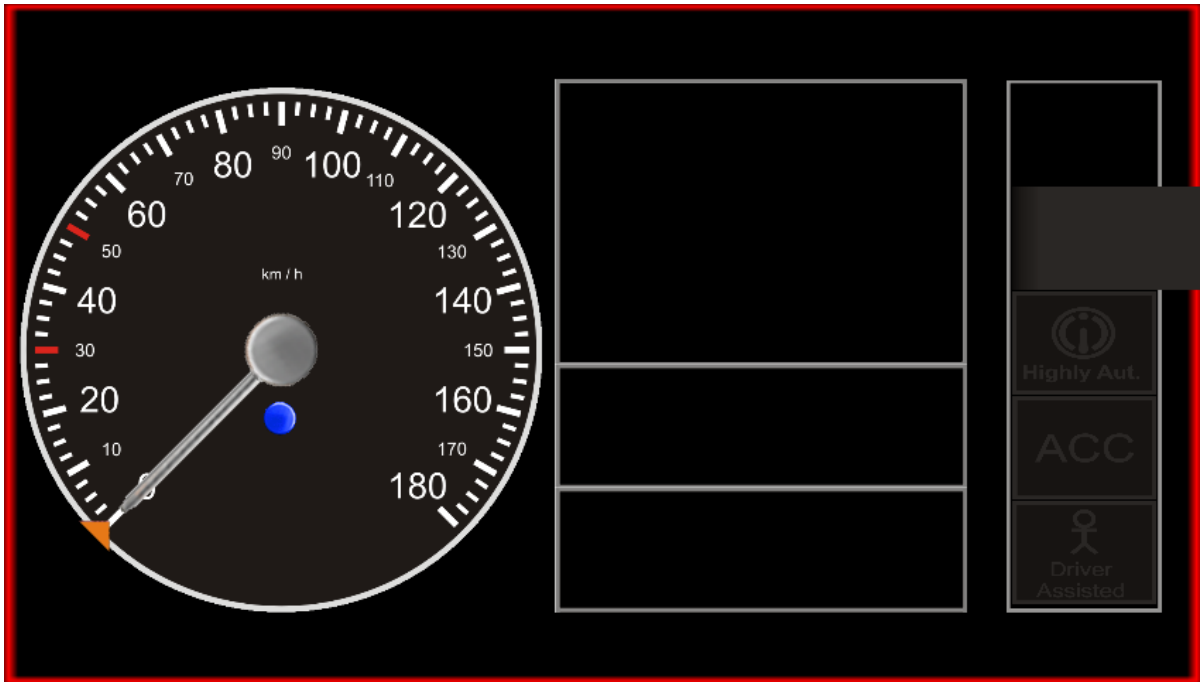
```
                Y = 50;
                W = 292;
                H = 89;
                Images = ("mmi\sp3000_2\ContainerWMPP.png");


            TextDisplay WarningMessagesL1
            {
                X = 0;
                Y = 44;
                W = 290;
                H = 30;
                Alignment = "center";
                Font = "verdana.ttf";
                FontColor = (0.91, 0.47, 0.09);
                Texts = ("Bitte übernehmen Sie!");
            };
        };

        ImageDisplay AutomMenu
        {
            X = 710;
            Y = 50;
            W = 110;
            H = 376;
            Images = ("mmi\sp3000_2\ACCMenu.png");

            ImageDisplay FullyAutomated
            {
                X = 4;
                Y = 228;
                W = 102;
                H = 73;
                Images = ("mmi\sp3000_2\FullyAut0.png");
            };
        };
};
```

This implies that the X and Y values of the individual containers (e.g., tachometer) can be derived based on the "global coordinate system". For example, WMPT with 390px in the X direction and AutoMenu with 710px in the X direction (and 50px in the Y direction).

The "sub-containers" ImageDisplays, Needles, etc. are dependent on the higher-level container. In a practical application, as shown in the WMPT example above, the text "WarningMessagesL1" is positioned relative to its parent container WMPT. Its position of 0px in the X direction and 44px in the Y direction results in being approximately in the center of the container (with a total height of 89px).

The width (W) and height (H) of the objects are important to some extent. For example, non-compliance with a specified size of a subordinate object that is allocated to a container may lead to the fact that a secondary object exceeds the limits of its higher-level container or other superordinate containers as well.

For example, changing width (W value) of the ImageDisplay FullyAutomated to 1,000px will make it wider than the AutoMenu (a higher-level container) that is 110px wide. Since this width exceeds that of other containers (here, it surpassed even the "top-level container MMI), the graphic overlaps the outer boundary (see illustration below).



The actual width/height is always identical with the selected width/height in the configuration and positioning is relative to the upper-level container.